

Sprint 3

Agentic AI Python Tutor — Neue Features



AGENT-GEDÄCHTNIS



CODE-REVIEW CHAIN



LERNPLAN-GENERATOR

AGENDA

02 / 07



01 — JOURNEY

Lisa's Lernstunde

Eine Studentin vom Login bis zum personalisierten Lernplan — alle neuen Features in einer echten Session.



02 — GEDÄCHTNIS

Kein Neustart mehr

Der Agent erinnert sich session-übergreifend — *"Letzte Woche haben wir Loops besprochen..."* ohne



03 — REVIEW

3-Ebenen-Feedback

Syntax ● · Stil ● ·
Architektur ● — jede Ebene separat, damit Lisa weiß, *was zuerst zu fixen ist.*



04 — LERNPLAN

Personalisiert

Plan basiert auf echten Skill-Scores — schwache Themen zuerst, *nicht eine generische To-do-Liste.*



05 — PERSISTENZ

Nichts geht verloren

Alle Chats jederzeit abrufbar — Lisa kann letzte Woche's Session aufmachen und weitermachen.



06 — DEMO

Live Demo

System in Aktion — alle neuen Sprint-3-Features live zeigen.

USER JOURNEY

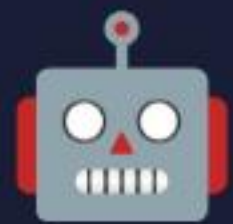
Lisa's Lernstunde — Eine Session von Anfang bis Ende



Lisa — Python-Anfängerin, 2. Semester. Sie hat letzte Woche Loops gelernt und will heute ihren ersten Code reviewen lassen.

**Dashboard**

Login → zwei klare Optionen: *KI Tutor* oder *Python Kurs*

`DashboardView.tsx`**Tutor startet**

"Hallo! 🖐️ *Ich bin dein KI-Tutor!*" — Roboter begrüßt sie mit Animation

`TutorTransition.tsx`**Tutor erinnert sich**

Agent: "*Letzte Woche haben wir Loops besprochen — weiter damit?*"

`memory_service.py`**Code Review**

Ihr Loop-Code bekommt **3 Ebenen** Feedback:
Syntax ●, Stil ●, Architektur ●

`RunnableSequence`**Lernplan**

Personalisierter Wochenplan: *schwache Skills zuerst*, max. 3 Skills pro Woche

`/learning-plan`**Nächste Woche**

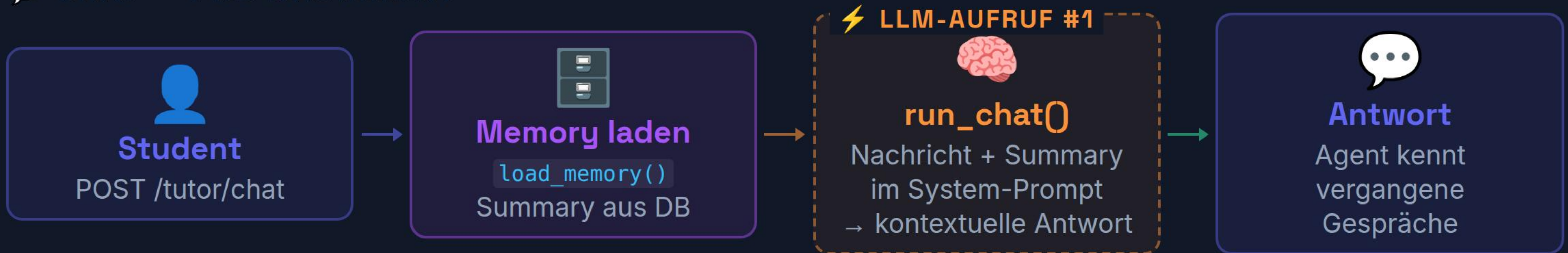
Lisa kommt zurück — Tutor weiß noch alles. *Kein Neustart.*

`memory_summary in D`

WIE FUNKTIONIERT'S? — AGENT-GEDÄCHTNIS

memory_service.py — Eigene Memory-Implementierung

💬 CHAT — PRO NACHRICHT



↓ Nach der Antwort: Summary aktualisieren

🧠 MEMORY-UPDATE — NACH JEDER NACHRICHT



⚡ 2 LLM-Aufrufe pro Nachricht

Aufruf 1: run_chat() — Antwort generieren mit

🧠 Kein roher Chat-Verlauf gespeichert

Summary statt History: LLM verdichtet den

WIE FUNKTIONIERT'S? — CODE-REVIEW CHAIN

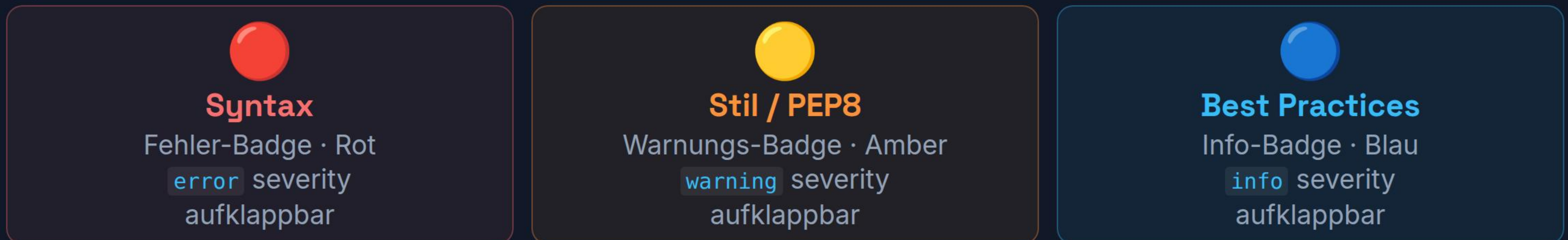
Code-Review Chain — 3 Schritte · 3 LLM-Aufrufe

🔗 GESAMTFLUSS — POST /TUTOR/REVIEW



↓ Frontend rendert strukturiertes Ergebnis

🖥️ FRONTEND — CODEREVIEWPANEL.TSX



🔗 Echte LangChain-Chain

⚡ Getrennte Verantwortlichkeiten

NÄCHSTER SPRINT

Was kommt als nächstes?

**Adaptiver Hinweis-Dialog**

Der Tutor erkennt, wann ein Lernender nicht weiterkommt, und bietet kontextbezogene Hinweise in drei Stufen an — ohne die Lösung direkt zu verraten.

☰ Stufe 1 · Stufe 2 · Lösung

Hint-on-Demand · progressiv · LLM-generiert

SPRINT 4

**Fehler-Erklärung auf Deutsch**

Python-Fehlermeldungen werden automatisch erkannt, übersetzt und auf Deutsch verständlich erklärt — ideal für deutschsprachige Lernende.

🔍 Error-Detection · DE-Übersetzung · Beispiel

Traceback-Parsing · LLM-Erklärung · kontextsensitiv

SPRINT 4

**Code-Umschreibungs-Tool**

Eingabe: funktionierender aber unschöner Code. Ausgabe: sauberer, pythonischer Code mit Erklärung der Verbesserungen — als Lernmaterial.

🔄 Diff-Ansicht · PEP8 · Idiomatic Python

vorher / nachher · inline Kommentare · LLM-refactor

SPRINT 4

Live Demo